

Assignment 1 — Problem setup

Agentic AI Workshop I

The problem: An alert fires: a service's error rate has crossed a threshold. Something has to gather evidence and decide one of three things.

Action	Cost when right	Cost when wrong
ignore	free	up to \$900 if it was a real outage
escalate to a human	\$25	\$25–\$150
rollback the last deploy	\$60	\$300–\$600 — irreversible

You have three tools: `read_logs`, `deploy_history`, `error_rate`. They occasionally time out. `score()` gives you accuracy and mean cost per alert. Cost is the metric that matters. There is a numeric shortcut that gets you to roughly 69%. Beating it means reading the log text, which is where a language model earns its place. **Use `alertgen.py`, unmodified.**

We will not build an agent this week. We will decide whether there should be one, and build the thing it has to beat. Assignments 1, 2, 3 solve this problem.

1. PEAS specification for the triage system. Your performance measure must contain a number.
2. Environment classification across the five properties, and one sentence per property on what it forces in your design.
3. The "should this be an agent?" question. Score it, and answer honestly. A runbook is a legitimate answer if you can defend it.
4. A decision tree for ignore / escalate / rollback on a single alert. Use the cost table above, estimate your own probabilities, and compute expected cost per branch.
5. Sensitivity. Find the probability at which your recommended action flips. State it as a number.
6. The baseline. Write a deterministic function — no model, no agent — that triages the first 100 alerts (`seed=0`). Report accuracy and cost per alert from `score()`.

Submit: notebook plus one page/markdown in the notebook. The page must state your flip point and which of your probability estimates you trust least.

Everyone's baseline will land around 65–70%. That number is your opponent for the next two assignments.